

研究进展

题目：面向代码变更的 Playwright 端到端针对性回归测试方法研究

0. 解决问题

解决 CI 中 Playwright E2E 测试的一个实际问题：全量跑太慢、只跑少了又怕漏、代码变了测试脚本还容易失效。

本研究的目标是：给定一次代码变更，自动产出一组“和本次变更相关、能运行、能覆盖变化”的 **Playwright E2E** 测试集。已有测试能复用就选择；选中的测试如果因页面变化失效就修复；已有测试覆盖不到的新变化再生成测试。

1. 为什么要做

现代 Web 项目提交频繁，CI 中每次都全量运行 E2E 测试成本较高。但 E2E 测试又很重要，因为它验证的是用户真实操作路径。问题是：

1. 全量运行贵：E2E 需要启动浏览器、访问页面、等待异步状态，执行时间和 CI 资源开销都比单元测试高。
2. 人工挑测试不可靠：开发者很难从一次 code diff 准确判断哪些 E2E 用例受影响。
3. 测试容易过时：页面结构、文本、定位器或断言一变，测试可能失败；失败不一定代表功能回归，也可能只是测试脚本过时。

所以工程上真正需要的不是“再生成一批测试”，而是一个面向 CI 的闭环：

对一次 **code diff**，只运行必要的 **E2E**；该修的修，该补的补，最后给出可执行的 **targeted E2E** 测试集。

2. 研究空白

我把相关工作分成三类来看：

1. 回归测试选择 / 变更影响分析

这类方法主要解决“代码变了，哪些测试要跑”，但成熟工作多集中在单元测试、集成测试或代码覆盖层面。Web E2E 测试和源码之间隔着页面、路由、组件、DOM、定位器，不能简单把单元测试 RTS 直接套过来。

2. E2E 测试生成

现有 Web/E2E 生成工作多从需求、页面或用户路径出发，目标是生成可执行测试。但它们通常不直接回答：“这次 diff 改了什么？缺哪条针对性测试？”

3. E2E 测试修复

现有修复工作主要关注定位器失效、等待问题、断言变化等。但多数是“测试失败后修测试”，没有把源码 diff 作为核心上下文，也没有服务于一个 targeted 回归测试集的整体可用性。

因此，本研究的切入点是：

建立 **code diff** 到 **Playwright E2E** 的关联：先选择受影响测试，再修复过时测试，最后为未覆盖的变更生成补充测试。

3. 具体做什么

输入

- 旧版本 `V_old`
- 新版本 `V_new`
- 本次代码变更 `diff`
- 项目已有的 Playwright E2E 测试集

输出

- 一组 targeted Playwright E2E 测试：
 - 已选择的相关旧测试；
 - 修复后可运行的旧测试；
 - 为未覆盖变更新生成的测试。
- 每条测试的选择 / 修复 / 生成依据。
- 执行结果、覆盖结果和成本报告。

成功标准

- 少跑：相比全量 E2E，执行数量和时间明显下降。
 - 不漏：受本次变更影响的测试尽量都被选中。
 - 能跑：最终进入 targeted set 的测试必须在新版本上真实执行通过。
 - 相关：生成的测试必须能触达本次 diff，而不是泛泛生成页面测试。
-

4. 方法设计

整体流程分为五步：

第一步：建立旧版本覆盖映射

在 `V_old` 上逐条运行已有 Playwright 测试，记录每条 E2E 测试实际触达的源码文件、路由和组件。

作用：给后续选择提供安全基础。也就是先知道“哪些测试过去碰过哪些代码”。

第二步：分析本次代码变更

解析 git diff，得到本次变更涉及的文件、函数、组件、路由和 UI 元素。

其中重点做一个 **Semantic UI Diff**：从 JSX/TSX 中抽取 tag、text、role、aria、testId、href、事件 handler 等语义 UI 节点，比较新旧版本，得到 UI 层面的新增、删除和修改。

第三步：选择相关 E2E 测试

选择规则是两个信号取并集：

1. 覆盖信号：旧版本中覆盖过变更文件 / 路由 / 组件的测试。
2. UI 语义信号：测试定位器和本次变更 UI 节点存在文本、testId、role、href 等语义关联。

覆盖信号负责尽量不漏；UI 语义信号负责在覆盖粒度较粗时提高选择精度。

第四步：运行、分诊和修复

在 `V_new` 上运行被选中的测试。

- 通过：直接进入 targeted set。
- 失败：判断是疑似真实回归，还是测试脚本过时。
- 如果是定位器、文本或断言过时，则结合 diff、DOM、trace 和 Semantic UI Diff 给出修复候选。
- 修复后的测试必须重新运行通过，才能进入 targeted set。

第五步：分析缺口并生成测试

如果本次 diff 中还有未被已选 / 已修测试触达的部分，则进入缺口生成。

生成不是“随便生成 E2E”，而是带约束生成：

- 必须触达本次 diff；
 - 必须包含可解释的操作路径；
 - 必须有和变化相关的断言；
 - 必须在 `V_new` 上真实运行通过。
-

5. 核心创新点

本研究不是单独做“测试选择”“测试生成”或“测试修复”，而是把三者放进同一个 code-diff 驱动的闭环里。

最关键的连接机制是 **Semantic UI Diff**:

- 在选择阶段，它把 UI 变更和测试定位器关联起来；
- 在修复阶段，它为失效定位器或断言提供候选；
- 在生成阶段，它指出哪些新增 UI 或变化路径还没有测试覆盖。

因此，它解决的是 E2E 场景里的核心断点：源码 **diff** 很难直接对应到浏览器中的用户操作和测试脚本。

6. 实验方法

RQ1: 能否少跑但不漏?

比较本方法和以下基线:

- 全量运行 retest-all;
- 随机选择同规模测试;
- 静态路由 / 组件启发式;
- 只用覆盖映射，不用 Semantic UI Diff。

主要指标:

- $\text{Reduction} = 1 - |\text{Sel}| / |S|$: 少跑多少;
- $\text{Safety} = |\text{Sel} \cap \text{Affected}| / |\text{Affected}|$: 受影响测试是否漏选;
- Precision : 选中的测试中有多少确实相关。

RQ2: diff 约束生成是否更相关?

比较:

- 无 diff 约束的页面 / 功能驱动生成;
- 本方法的 diff 缺口生成。

主要看:

- 是否触达 diff;
- 是否可执行;
- 人工检查语义上是否是有效测试。

RQ3: 修复是否提升 **targeted set** 可用性?

比较:

- 不修复；
- 纯属性 self-healing；
- 不带 diff 上下文的 LLM 修复；
- 本方法的 diff + DOM + 语义候选修复。

主要看最终 targeted set 中有多少测试可以实际运行并覆盖变化。

RQ4：该方法在真实项目中的运行成本和工程可落地性如何？

评估：

- Token 消耗
- 平均修复/生成迭代次数
- 端到端耗时（Wall-clock Time）
- 单个 PR 的 CI 等待时间（CI Latency）
- 不同 Playwright 项目上的泛化能力

7. 当前进展

已经完成一个端到端原型 DiffE2E，覆盖以下能力：

- 旧版本 E2E 覆盖映射；
- diff 分析；
- 双信号测试选择；
- 运行与失败分诊；
- 定位器 / 断言修复；
- 缺口生成；
- 执行验证和结果报告。

受控实验结果

在一个受控多文件应用的 24 个真实 git 变更过渡上：

方法	测试执行量	是否漏测	结果
Retest-All	100%	否	安全但成本最高
Random-K	37.5%	严重	不可用
静态启发式	12.5%	存在	风险较高
本方法	37.5%	否	兼顾效率与安全性

在 24 个真实变更过渡的预实验中，本方法仅执行约 37.5% 的测试（执行量降低 62.5%），同时未漏掉任何受影响测试（Safety=1.0）。

小型真实工程验证

在 React + Vite + Playwright 工程中，Semantic UI Diff 能把覆盖信号的粗粒度选择进一步收窄：覆盖信号会选 3 条测试，UI 语义信号能定位到 1 条更相关的测试，并完成后续修复和新增按钮测试生成。

真实数据预实验

对 ReproBreak 相关数据做了离线分析，说明 Semantic UI Diff 对一部分定位器变化可以直接给出确定性修复线索。

8. 当前结论

已有实验支持该方法的可行性和潜在收益；最终有效性与泛化性需要通过真实项目多 commit replay 和真实 LLM 实验进一步验证。

9. 下一步计划

优先级	任务	目的
高	跑真实项目多 commit replay	验证 RQ1 在真实 Playwright 项目上的少跑和不漏
高	接入真实 LLM，并与规则 / stub 对比	验证生成和修复是否真正有语义收益
中	完成 ReproBreak 449 个可复现定位器失效的端到端修复实验	补强 RQ3
中	扩大变更类型和样本规模	提高统计可信度
低	做 GitHub Actions 集成 demo	展示工程落地形态

10. 参考文献

A 类：核心文献（直接对标）

[1] UTFix: Change Aware Unit Test Repairing using LLM. Proc. ACM Program. Lang. (OOPSLA), 2025. <https://arxiv.org/abs/2503.14924>

[2] Code-Aware Prompting (SymPrompt): Coverage-Guided Test Generation in Regression Setting using LLM. Proc. ACM Softw. Eng. (FSE), 2024. <https://arxiv.org/abs/2402.00097>

[3] Can LLM Generate Regression Tests for Software Commits? arXiv:2501.11086, 2025. <https://arxiv.org/abs/2501.11086>

- [4] Testora: Using Natural Language Intent to Detect Behavioral Regressions. arXiv:2503.18597, 2025. <https://arxiv.org/abs/2503.18597>
- [5] AI for Context-Aware Visual Change Detection in Software Test Automation. Progress in Artificial Intelligence, 2024. <https://arxiv.org/abs/2405.00874>
- [6] Feature-Driven End-to-End Test Generation (AutoE2E). ICSE, 2025. <https://arxiv.org/abs/2408.01894>
- [7] Automated Web Application Testing: E2E Test Case Generation with LLMs and Screen Transition Graphs. arXiv:2506.02529, 2025. <https://arxiv.org/abs/2506.02529>
- [8] Scenario-Guided LLM-based Mobile App GUI Testing. ACM TOSEM, 2025. <https://arxiv.org/abs/2506.05079>
- [9] GenIA-E2ETest: A Generative AI-Based Approach for End-to-End Test Automation. SBES, 2025. <https://arxiv.org/abs/2510.01024>
- [10] Practical Limits of Autonomous Test Repair: A Multi-Agent Case Study (LLM + LangGraph + Playwright). arXiv:2605.01471, 2026. <https://arxiv.org/abs/2605.01471>
- [11] Automated Test Case Repair Using Language Models (TaRGET). IEEE Transactions on Software Engineering, 2025. <https://arxiv.org/abs/2401.06765>
- [12] Semantic Test Repair for Web Applications. ESEC/FSE, 2023. <https://doi.org/10.1145/3611643.3616324>
- [13] WEFix: Automatic Generation of Explicit Waits for Web E2E Flaky Tests. The Web Conf (WWW), 2024. <https://arxiv.org/abs/2402.09745>
- [14] Fix the Tests: Augmenting LLMs to Repair Test Cases with Static Collector and Neural Reranker. ISSRE, 2024. <https://arxiv.org/abs/2407.03625>
- [15] Unit Test Update through LLM-Driven Context Collection and Error-Type-Aware Refinement. ASE, 2025. <https://arxiv.org/abs/2509.24419>
- [16] Understanding & Enhancing Attribute Prioritization in Fixing Web UI Tests with LLMs (Guiding ChatGPT to Fix Web UI Tests). ICST, 2025. <https://arxiv.org/abs/2312.05778>
- [17] Time-based Repair for Asynchronous Wait Flaky Tests in Web Testing. arXiv:2305.08592, 2023. <https://arxiv.org/abs/2305.08592>
- [18] Towards Predicting Fragility in End-to-End Web Tests. EASE, 2024. <https://doi.org/10.1145/3661167.3661179>

B 类：支撑文献（方法论与背景）

- [19] Efficient Incremental Code Coverage Analysis for Regression Test Suites (iJaCoCo). ASE, 2024. <https://arxiv.org/abs/2410.21798>
- [20] More Precise Regression Test Selection via Reasoning about Semantics-Modifying Changes. ISSTA, 2023. <https://doi.org/10.1145/3597926.3598086>
- [21] Test Selection for Unified Regression Testing. ICSE, 2023. <https://doi.org/10.1109/ICSE48619.2023.00145>
- [22] Change Impact Analysis in Microservice Systems: A Systematic Literature Review. Journal of Systems and Software, 2024. <https://doi.org/10.1016/j.jss.2024.112241>

[23] Datalog-Based Language-Agnostic Change Impact Analysis for Microservices. ICSE, 2025. <https://doi.org/10.1109/ICSE55347.2025.00115>

[24] Hybrid Regression Test Selection by Integrating File and Method Dependences. ASE, 2024. <https://doi.org/10.1145/3691620.3695525>

[25] Practical Pipeline-Aware Regression Test Optimization for CI. ICST, 2025. <https://arxiv.org/abs/2501.11550>

[26] Regression Test Selection in Test-Driven Development. Automated Software Engineering, 2023. <https://doi.org/10.1007/s10515-023-00405-w>

[27] An Empirical Evaluation of Using LLMs for Automated Unit Test Generation. IEEE Transactions on Software Engineering, 2023. <https://arxiv.org/abs/2302.06527>

[28] CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained LLMs. ICSE, 2023. <https://doi.org/10.1109/ICSE48619.2023.00085>

[29] ChatUniTest: A Framework for LLM-Based Test Generation. FSE Companion, 2023. <https://arxiv.org/abs/2305.04764>

[30] Effective Test Generation Using Pre-trained LLMs and Mutation Testing. Information and Software Technology, 2023. <https://arxiv.org/abs/2308.16557>

[31] ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. IEEE Transactions on Software Engineering, 2023. <https://arxiv.org/abs/2307.00588>

[32] CAT-LM: Training Language Models on Aligned Code And Tests. ASE, 2023. <https://arxiv.org/abs/2310.01602>

[33] SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. NeurIPS, 2024. <https://arxiv.org/abs/2406.12952>

[34] Automated Unit Test Improvement using LLMs at Meta (TestGen-LLM). FSE Companion, 2024. <https://arxiv.org/abs/2402.09171>

[35] LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities. QRS, 2023. <https://arxiv.org/abs/2309.13574>

[36] GAMMA: Revisiting Template-Based APR via Mask Prediction. ASE, 2023. <https://arxiv.org/abs/2309.09308>

[37] FlakyFix: LLMs for Predicting Flaky Test Fix Categories and Test Code Repair. IEEE Transactions on Software Engineering, 2024. <https://arxiv.org/abs/2307.00012>

[38] StubCoder: Automated Generation and Repair of Stub Code for Mock Objects. ACM TOSEM, 2023. <https://arxiv.org/abs/2307.14733>

[39] Effortless Test Maintenance: A Critical Review of Self-Healing Frameworks. IJRASET, 2023. <https://doi.org/10.22214/ijraset.2023.56048>

[40] A Multi-Year Grey Literature Review on AI-assisted Test Automation. Information and Software Technology, 2024. <https://arxiv.org/abs/2408.06224>

[41] Exploring the Integration of LLMs in Industrial Test Maintenance Processes. arXiv:2409.06416, 2024. <https://arxiv.org/abs/2409.06416>

[42] Software Testing With Large Language Models: Survey, Landscape, and Vision. IEEE Transactions on Software Engineering, 2023. <https://arxiv.org/abs/2307.07221>

[43] A Survey on Web Testing: On the Rise of AI and Applications in Industry. arXiv:2503.05378, 2025. <https://arxiv.org/abs/2503.05378>

[44] Challenges of End-to-End Testing with Selenium WebDriver and How to Face Them: A Survey. ICST, 2023. <https://doi.org/10.1109/ICST57152.2023.00039>

[45] Vision-Based Mobile App GUI Testing: A Survey. ACM Computing Surveys, 2023. <https://arxiv.org/abs/2310.13518>

[46] A Comprehensive Survey of AI-Driven Advancements in Automated Program Repair and Code Generation. arXiv:2411.07586, 2024. <https://arxiv.org/abs/2411.07586>

[47] Observation-Based Unit Test Generation at Meta (TestGen). FSE Companion, 2024. <https://arxiv.org/abs/2402.06111>

[48] E2EGit: A Dataset of End-to-End Web Tests in Open Source Projects. MSR, 2025. <https://doi.org/10.1109/MSR66628.2025.00121>

C 类：最新前沿 (2025–2026)

[49] Change And Cover (ChaCo): Last-Mile, Pull Request-Based Regression Test Augmentation. arXiv:2601.10942, 2026. <https://arxiv.org/abs/2601.10942>

[50] PR-Aware Automated Unit Test Generation: Challenges and Opportunities. arXiv:2605.25285, 2026. <https://arxiv.org/abs/2605.25285>

[51] Just-in-Time Catching Test Generation at Meta. arXiv:2601.22832, 2026. <https://arxiv.org/abs/2601.22832>

[52] TestWeaver: Execution-aware, Feedback-driven Regression Testing Generation with LLMs. arXiv:2508.01255, 2025. <https://arxiv.org/abs/2508.01255>

[53] Evaluating LLM-Based Test Generation Under Software Evolution. arXiv:2603.23443, 2026. <https://arxiv.org/abs/2603.23443>

[54] Code-A1: Adversarial Evolving of Code LLM and Test LLM via RL. arXiv:2603.15611, 2026. <https://arxiv.org/abs/2603.15611>

[55] ReproBreak: A Dataset of Reproducible Web Locator Breaks. arXiv:2605.12158, 2026. <https://arxiv.org/abs/2605.12158>

[56] FlakyGuard: Automatically Fixing Flaky Tests at Industry Scale. ASE, 2025. <https://arxiv.org/abs/2511.14002>

[57] YATE: The Role of Test Repair in LLM-Based Unit Test Generation. arXiv:2507.18316, 2025. <https://arxiv.org/abs/2507.18316>

[58] Hierarchical Knowledge Injection for Improving LLM-based Program Repair. ASE, 2025. <https://arxiv.org/abs/2506.24015>

[59] VISCA: Inferring Component Abstractions for Automated End-to-End Testing. arXiv:2506.04161, 2025. <https://arxiv.org/abs/2506.04161>

[60] ViMoTest: Specify ViewModel-Based GUI Test Scenarios using Projectional Editing. ICST, 2025. <https://arxiv.org/abs/2504.16753>

[61] Automated Functional Testing for Malleable Mobile Application Driven from User Intent. arXiv:2604.02079, 2026. <https://arxiv.org/abs/2604.02079>

[62] Android Instrumentation Testing in CI: Practices, Patterns, and Performance. arXiv:2604.03438, 2026. <https://arxiv.org/abs/2604.03438>

[63] Do Autonomous Agents Contribute Test Code? A Study of Tests in Agentic Pull Requests. arXiv:2601.03556, 2026. <https://arxiv.org/abs/2601.03556>

[64] Names Are All You Need (NameRTS): Effective and Safe Regression Test Selection for Python. arXiv:2605.25356, 2026. <https://arxiv.org/abs/2605.25356>

[65] Formalizing Regression Testing for Agile and CI Environments. arXiv:2511.02810, 2025. <https://arxiv.org/abs/2511.02810>

[66] Understanding Automated Program Repair Agents Through the Lens of Traceability: An Empirical Study. arXiv:2506.08311, 2025. <https://arxiv.org/abs/2506.08311>
